

Cahier de recette

Sujet : Développement d'un Assistant Pédagogique

Groupe L3S

Auteurs : Ines Yousfi
Mounir Lamouchi
Rayane Kachbi

Encadrant : David Janiszek

Version du document : 1.0

Confidentialité : Réservé aux étudiants UFR Maths-Info Paris Descartes.

Type de diffusion : Document électronique (.pdf)

Sommaire

1	Introduction	2
1.1	Objectifs & Méthodes	2
1.2	Documents de références	2
2	Guide de lecture	3
2.1	Maîtrise d'œuvre	3
2.2	Maîtrise d'ouvrage	3
3	Concepts de base	3
3.1	Architecture RAG (Retrieval-Augmented Generation)	3
3.2	Vectorisation et embeddings	4
3.3	Jeu de données de test	4
4	Description de la fourniture	4
4.1	Livraison du code source	4
4.2	Mise à disposition de l'application	5
4.3	Documentation	5
5	Moyens d'essai et outils	5
5.1	Environnement de test	5
5.2	Outils de test	6
6	Conformité aux spécifications générales	6
7	Conformité aux spécifications fonctionnelles & d'interface	8
7.1	Fonctionnalités principales	8
7.2	Fonctionnalités secondaires	9
7.3	Fonctionnalités facultatives	9
8	Conformité de la documentation	9
9	Annexes	10
10	Glossaire	12
	Références	13
	Index	14

1 Introduction

Ce document constitue le cahier de recettes du projet L3S1, portant sur le développement de l'application web « Assistant Pédagogique » (AP-UP). Son objectif est de définir rigoureusement l'ensemble des protocoles de tests permettant de vérifier que le logiciel développé répond de manière exhaustive aux exigences fonctionnelles, techniques et de sécurité définies dans le cahier des charges.

Le présent cahier établit les objectifs des tests et la méthodologie de vérification. Il inclut un guide de lecture, rappelle les concepts fondamentaux du système, et liste les moyens d'essais mis en place.

Enfin, nous allons détailler les scénarios de tests applicables à chaque module (ingestion, vectorisation, génération, filtrage), en précisant les critères d'acceptabilité stricts à respecter.

1.1 Objectifs & Méthodes

L'objectif principal de ce cahier de recettes est de garantir que l'assistant pédagogique fournit des réponses fiables, sourcées, et qu'il fonctionne de manière fluide pour les étudiants comme pour les enseignants. Pour y parvenir, notre méthodologie s'appuie sur plusieurs niveaux de tests :

- **Tests unitaires** : Ils permettront de vérifier le bon fonctionnement individuel et isolé de chaque composant technique du backend (découpe des documents, connexion à la base de données vectorielle, exécution des requêtes ORM).
- **Tests d'intégration** : Ils ont pour objectif de s'assurer que les différents modules communiquent correctement entre eux, de l'ingestion d'un document PDF jusqu'à l'interrogation de l'API Pléiade.
- **Tests fonctionnels et de robustesse** : Ils visent à valider les fonctionnalités de l'interface utilisateur et à éprouver les filtres de sécurité du système en soumettant des requêtes complexes, hors-sujet ou ambiguës (tests anti-hallucinations).
- **Tests utilisateurs (Ergonomie)** : Ils évalueront la facilité de prise en main des deux espaces de l'application (étudiant et administrateur) lors de sessions de simulation.

Chaque test sera consigné sous forme de scénario, décrivant la procédure à suivre, le résultat attendu et le statut de validation.

1.2 Documents de références

L'élaboration de ce cahier de recettes et la définition des cas de tests s'appuient sur les documents suivants :

- Le Cahier des Charges du projet.
- Les annexes (modèle de cahier de recettes disponible sur le site du projet).
- Les documentations techniques officielles des outils intégrés.
- Les comptes rendus des réunions hebdomadaires avec la maîtrise d'ouvrage.

2 Guide de lecture

Ce document a pour but de décrire la livraison de l'ensemble des éléments du projet. Il précise également les différents tests de conformité du projet ainsi que le planning de ces derniers.

Il s'adresse aux développeurs, aux chefs de projet, et à toute partie prenante souhaitant comprendre les objectifs et les contraintes du projet. Ce guide explique comment utiliser le cahier de recettes selon le rôle de chaque membre du projet :

- **Développeurs** : utilisent ce document pour s'assurer que toutes les fonctionnalités sont correctement implémentées.
- **Testeurs** : suivent les procédures de test pour vérifier que le logiciel fonctionne correctement et signaler d'éventuels bugs.
- **Encadrant** : s'appuie sur ce document pour évaluer la qualité du projet et son respect des exigences initiales.

2.1 Maîtrise d'œuvre

La maîtrise d'œuvre est effectuée par les membres du groupe L3S1. Ce document doit nous servir de guide pour la suite du projet. Nous devons comprendre les attentes de l'encadrant et établir un plan pour y répondre au mieux.

2.2 Maîtrise d'ouvrage

La maîtrise d'ouvrage représente l'encadrant de ce projet David Janiszek et les éventuels utilisateurs finaux. Grâce à ce cahier de recettes, ils peuvent vérifier que le projet est conforme aux attentes et identifier les points à améliorer.

3 Concepts de base

Cette section introduit les concepts de base du projet, servant ainsi à la compréhension globale du cahier de recettes.

3.1 Architecture RAG (Retrieval-Augmented Generation)

Notre application web repose sur une architecture de type RAG (Retrieval Augmented Generation). Cette approche consiste à combiner un système de recherche d'informations dans une base documentaire avec un modèle de langage capable de générer des réponses en langage naturel.

Lorsqu'un utilisateur pose une question, le système recherche d'abord les passages les plus pertinents dans les documents pédagogiques disponibles. Ces informations sont ensuite transmises au modèle de langage, qui génère une réponse basée sur le contenu de ces documents. Cette architecture permet de limiter les réponses générées aux informations issues des supports de cours fournis par les enseignants.

3.2 Vectorisation et embeddings

Afin de permettre la recherche sémantique dans les documents, le contenu textuel est transformé en embeddings, ou en vecteurs numériques représentant le sens du texte grâce au processus de vectorisation.

Cette représentation vectorielle permet de comparer la similarité entre une requête utilisateur et les passages présents dans les documents, afin d'identifier les contenus les plus pertinents.

3.3 Jeu de données de test

Un jeu de données de test (ou Dataset de requêtes) correspond à un ensemble de documents ou de requêtes utilisés pour évaluer le comportement du système lors des phases de validation.

Dans notre projet, ces données permettront notamment de vérifier la pertinence des réponses générées par l'Assistant Pédagogique, ainsi que sa capacité à traiter différents types de questions.

4 Description de la fourniture

Dans le cadre de notre projet, la fourniture finale correspond à la mise à disposition de l'application AP-UP (Assistant Pédagogique) ainsi que de l'ensemble des éléments nécessaires à son utilisation, son déploiement et sa maintenance.

Le produit principal livré est une application web permettant aux étudiants d'interagir avec un assistant conversationnel capable de répondre aux questions à partir des supports de cours fournis par les enseignants.

L'application comporte deux espaces principaux :

- **Un espace étudiant** : accessible après authentification, permettant de poser des questions via une interface conversationnelle de type chatbot et d'obtenir des réponses basées sur les documents pédagogiques fournis par les enseignants.
- **Un espace administrateur (enseignant)** : permettant de déposer et gérer les supports de cours, de supprimer des fichiers, et de modifier le prompt système.

L'ensemble du projet sera livré sous la forme de trois éléments principaux : le code source, l'application web déployée, et la documentation associée.

4.1 Livraison du code source

Le code source complet de l'application sera versionné et mis à disposition via un dépôt SVN hébergé sur La Forge. L'utilisation de ce système de gestion de versions permet d'assurer le suivi des modifications, la traçabilité du développement et la maintenance du projet.

Ce dépôt contiendra l'ensemble des fichiers nécessaires au fonctionnement du projet, notamment :

- le code source de l'application (backend et interface),

- les scripts nécessaires à l'installation et au déploiement,
- les fichiers de configuration du projet.

4.2 Mise à disposition de l'application

L'application sera livrée sous la forme d'une application web fonctionnelle accessible via un navigateur internet.

Les utilisateurs pourront accéder directement à l'application en se connectant à l'adresse fournie. Après authentification, les utilisateurs pourront interagir avec l'assistant pédagogique.

Cette approche permet une utilisation simple, centralisée et accessible depuis différents environnements.

4.3 Documentation

L'ensemble de la documentation du projet sera fourni au format PDF et déposé dans l'onglet Documents du projet sur La Forge.

Les documents livrés sont les suivants :

- **Cahier des charges** : décrit les besoins, les objectifs et les contraintes du projet. Il constitue le document de référence pour le développement.
- **Cahier de recettes** : présente les cas de tests permettant de vérifier la conformité de l'application aux exigences définies.
- **Conception générale** : décrit l'architecture globale du système et les principaux choix techniques réalisés.
- **Conception détaillée** : présente le fonctionnement interne des différents composants du système ainsi que les diagrammes UML associés.
- **Manuel d'utilisation** : guide destiné aux utilisateurs finaux (étudiants et enseignants) expliquant comment utiliser l'application.
- **Manuel d'installation** : documentation technique décrivant les étapes nécessaires au déploiement de l'application.
- **Plan de tests** : décrit la stratégie de tests adoptée et les méthodes utilisées pour valider le bon fonctionnement du système.
- **Documentation interne du code** : documentation générée à partir des commentaires et docstrings présents dans le code source, destinée aux développeurs.
- **Rapport final de projet** : document de synthèse présentant le déroulement du projet, les choix techniques réalisés et les perspectives d'évolution.

5 Moyens d'essai et outils

5.1 Environnement de test

Les tests seront réalisés dans un environnement reproduisant l'architecture technique prévue pour l'application. Les principales technologies utilisées sont :

- **Langage de programmation** : Python 3.10+ avec environnement virtuel (uv package manager).
- **Interface utilisateur (UI)** : librairie Chainlit de Python.
- **Base de données** : PostgreSQL avec l'extension pgvector.
- **Serveur d'inférence** : API Pléiade et Ollama pour l'exécution des modèles de langage.

5.2 Outils de test

- **pytest** : Framework Python pour l'exécution des tests unitaires et d'intégration sur les modules backend (ingestion, vectorisation, recherche, génération, filtrage).
- **Postman / curl** : Pour tester les appels API (API Pléiade, endpoint Chainlit) de manière isolée.
- **Navigateur web** : Pour exécuter les tests fonctionnels et d'interface (espace étudiant et espace administrateur) en conditions réelles d'utilisation.
- **Jeux de données de test** : Un jeu de données documentaire de test, composé de supports de cours fictifs ou réels, sera utilisé pour alimenter la base vectorielle lors des tests. Ces fichiers couvriront différents formats de contenu (texte pur, formules mathématiques, tableaux, images) et différents types de fichiers (PDF, PPTX...).
- **Dataset de requêtes de test** : Un jeu de questions de test préétablies, couvrant des cas nominaux, des cas limites (questions hors-programme, requêtes vides, formulations ambiguës) et des cas d'erreur, sera utilisé pour évaluer la précision et la robustesse du système.
- **Mesure des performances** : Pour vérifier le respect des critères de performance, notamment le temps de réponse de l'assistant.
- **La Forge (SVN)** : Utilisée pour assurer la traçabilité des versions testées et documenter les problèmes détectés.

6 Conformité aux spécifications générales

Cette partie a pour objectif de vérifier que l'application développée correspond bien aux exigences définies dans le cahier des charges du projet.

La phase de recette permet de s'assurer que les différents éléments livrés (application web, code source et documentation) fonctionnent correctement et répondent aux besoins des utilisateurs. Pour cela, les différents tests présentés dans ce document seront réalisés afin de vérifier le comportement du système.

Plusieurs points seront vérifiés lors de cette phase :

- **Fonctionnalités de l'application** : Vérifier que les principales fonctionnalités prévues sont bien disponibles. Par exemple, les étudiants doivent pouvoir poser des questions à l'assistant via l'interface conversationnelle, et les enseignants doivent pouvoir ajouter ou supprimer des documents pédagogiques dans l'espace administrateur.
- **Fonctionnement du système RAG** : Vérifier que les différentes étapes du système fonctionnent correctement : ingestion des documents, création des embed-

dings, recherche dans la base de données et génération des réponses par le modèle de langage.

- **Pertinence des réponses de l'assistant** : S'assurer que les réponses générées par l'assistant sont compréhensibles et qu'elles s'appuient sur les documents pédagogiques fournis.
- **Robustesse du système** : Tester le comportement de l'application avec différents types de questions (questions normales, questions hors sujet, requêtes vides ou ambiguës) afin de vérifier que le système reste stable et d'éviter toute hallucination.
- **Accès à l'application** : Vérifier que l'application est accessible depuis un navigateur web et que les différentes fonctionnalités de l'interface fonctionnent correctement.
- **Performance de l'application** : Vérifier que le temps de réponse de l'assistant reste raisonnable afin de garantir une bonne expérience utilisateur.

À la fin de cette phase de recette, une décision sera prise concernant la validation du projet. Le décisionnaire final est M. David Janiszek, encadrant du projet. Il vérifiera que les tests décrits dans ce cahier de recettes ont été réalisés et que les résultats sont satisfaisants.

Si les tests sont validés et que l'application fonctionne correctement, le projet sera accepté. Dans le cas contraire, des corrections pourront être demandées avant la validation finale.

7 Conformité aux spécifications fonctionnelles & d'interface

7.1 Fonctionnalités principales

ID	Description	Contraintes	Dép.	Action (Procédure)	Résultat attendu	Validé ?
FP-01	Accès à l'application web	Navigateur compatible (Chrome, Firefox)	Aucune	Ouvrir l'adresse web de l'application.	L'interface se charge correctement sans erreur.	☐ Oui ☐ Non
FP-02	Authentification utilisateur/administrateur	Compte valide	FP-01	Entrer les identifiants et valider.	L'utilisateur/administrateur est connecté et accède à son espace.	☐ Oui ☐ Non
FP-03	Dépôt de documents pédagogiques	Fichier valide	FP-02	Aller dans l'espace administrateur et uploader un fichier.	Le document est accepté et ajouté à la liste des fichiers.	☐ Oui ☐ Non
FP-04	Indexation automatique des documents	Document correctement uploadé	FP-03	Vérifier le statut du document importé.	Le système découpe le document, génère les embeddings et les stocke dans la base de données vectorielle.	☐ Oui ☐ Non
FP-05	Pose de question par un étudiant	Interface de chat fonctionnelle	FP-02	Écrire une question dans la barre de chat et l'envoyer.	La question s'affiche et l'interface indique que le modèle « réfléchit ».	☐ Oui ☐ Non
FP-06	Recherche sémantique	Base pgvector active	FP-05	Soumettre une question spécifique au cours.	Le système identifie et récupère en arrière-plan les passages pertinents.	☐ Oui ☐ Non
FP-07	Génération de réponse par le modèle	API Pléiade/Ollama active	FP-06	Attendre la fin du processus de génération.	L'assistant formule une réponse cohérente basée sur les documents.	☐ Oui ☐ Non
FP-08	Filtrage de sécurité des requêtes	Système de filtrage/SLM actif	FP-05	Poser une question hors-sujet ou inappropriée.	La requête est filtrée ou bloquée, un message indique que l'IA ne répond qu'au cours.	☐ Oui ☐ Non
FP-09	Affichage de la réponse	Interface Chainlit fonctionnelle	FP-07	Observer le retour de l'assistant suite à la question posée.	La réponse s'affiche correctement dans la conversation.	☐ Oui ☐ Non

7.2 Fonctionnalités secondaires

ID	Description	Contraintes	Dép.	Action (Procédure)	Résultat attendu	Validé ?
FS-01	Gestion des rôles	Comptes configurés	FP-02	Se connecter successivement avec un compte Étudiant puis Enseignant.	L'étudiant n'a accès qu'au chat ; l'enseignant a accès au panneau d'administration.	☐ Oui ☐ Non
FS-02	Mise en cache des réponses	Système de cache actif	FP-07	Poser une question, chronométrer, puis reposer exactement la même question.	La seconde réponse est envoyée plus rapidement.	☐ Oui ☐ Non
FS-03	Suppression d'un document	Document existant	FP-03	Depuis l'espace administrateur, sélectionner un document, cliquer sur « Supprimer ».	Le document et ses vecteurs (embeddings) associés disparaissent du système.	☐ Oui ☐ Non
FS-04	Gestion des erreurs de saisie	Interface active	FP-05	Envoyer une requête vide ou avec uniquement des espaces.	Le système refuse l'envoi et demande de formuler une vraie question.	☐ Oui ☐ Non
FS-05	Affichage des sources utilisées	Traçabilité des passages active	FP-07	Inspecter la réponse générée par l'IA.	L'interface affiche en bas de la réponse les références exactes utilisées (nom du PDF, page).	☐ Oui ☐ Non

7.3 Fonctionnalités facultatives

ID	Description	Contraintes	Dép.	Action (Procédure)	Résultat attendu	Validé ?
FF-01	Export des réponses	Fonctionnalité d'export disponible	FP-09	Cliquer sur le bouton d'export à côté d'une réponse.	La réponse est téléchargée sur l'ordinateur de l'utilisateur.	☐ Oui ☐ Non
FF-02	Mise en cache globale (inter-utilisateurs)	Système de cache global actif, 2 comptes distincts.	FP-07	Étudiant A pose une question. Depuis un autre navigateur, Étudiant B pose exactement la même question.	La réponse est fournie instantanément à l'Étudiant B sans nouvel appel au modèle IA.	☐ Oui ☐ Non

8 Conformité de la documentation

L'ensemble de la documentation livrée pour le projet doit être claire et rigoureusement à jour, elle s'adresse à la fois aux utilisateurs finaux et aux futurs mainteneurs.

La conformité de la documentation sera évaluée selon les livrables suivants :

- **Manuel d'utilisation** : Il doit détailler les fonctionnalités de l'application. Ce guide doit accompagner l'enseignant (espace administrateur) dans le dépôt de ses supports de cours et la gestion du prompt système, et guider l'étudiant dans l'utilisation de l'interface conversationnelle (intégration de captures d'écran exigée).

- **Manuel d'installation** : Il doit décrire de manière simple et claire la procédure pour récupérer le projet via le dépôt SVN, ainsi que les instructions basiques pour lancer l'environnement (installation des dépendances Python et lancement de l'interface).
- **Documents de conception (Générale et Détaillée)** : Ils doivent présenter l'architecture logicielle finale, incluant les schémas UML, le modèle de données, et le pipeline RAG (ingestion, vectorisation, génération, filtrage).
- **Rapport Final** : Il doit justifier les choix techniques finaux, et dresser le bilan des tests de conformité (reprenre le tableau du cahier des charges et le compléter).

Critères de validation pour toute la documentation :

- **Structure et lisibilité** : Chaque document technique doit comporter une table des matières, une pagination, un glossaire pour les termes spécifiques, et un index.
- **Exactitude et traçabilité** : Les informations décrites doivent correspondre strictement aux fonctionnalités réelles de l'application livrée sur La Forge (SVN). Toute fonctionnalité abandonnée ou modifiée en cours de développement ne doit plus figurer dans la documentation finale.

9 Annexes

Annexe 1 : Interface Utilisateur (Chainlit)

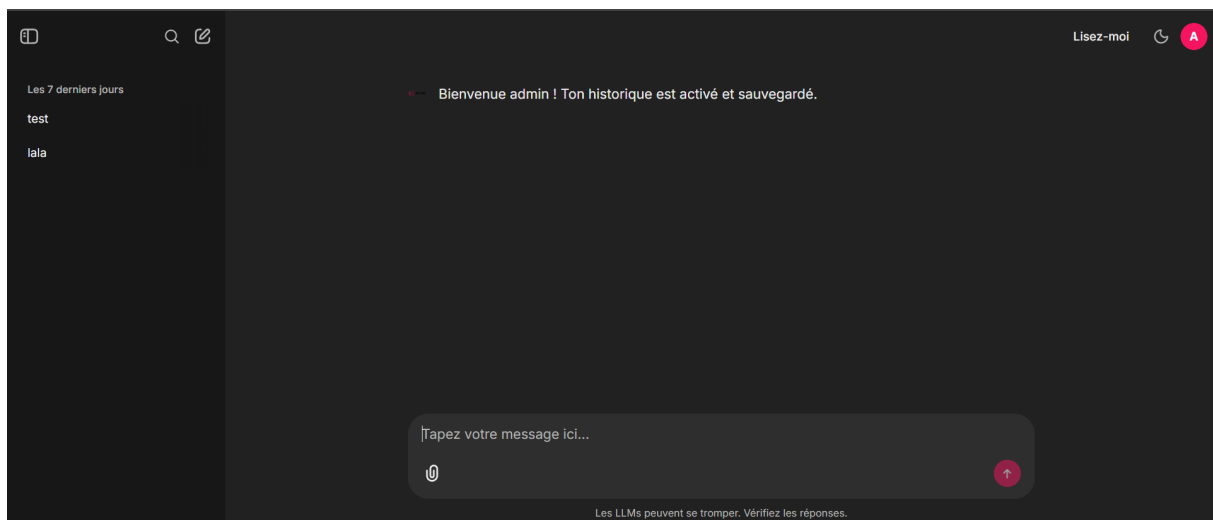


FIGURE 1 – Aperçu de l'interface conversationnelle (Chainlit) en mode administrateur. Cette interface sera utilisée pour valider les tests d'interaction (FP-05, FP-09).

Annexe 2 : Versioning et Centralisation du Code (SVN)

root

Nom	Taille	Révision	Âge	Auteur	Commentaire
branches		1	environ un mois	Rayane Kachbi	First commit. Création de branches, tags, trunk.
tags		1	environ un mois	Rayane Kachbi	First commit. Création de branches, tags, trunk.
trunk		1	environ un mois	Rayane Kachbi	First commit. Création de branches, tags, trunk.

Dernières révisions

#	Date	Auteur	Commentaire
1	04/02/2026 12:32	Rayane Kachbi	First commit. Création de branches, tags, trunk.

[Voir toutes les révisions](#)

FIGURE 2 – Dépôt SVN initialisé sur La Forge, garantissant le suivi des versions et le travail collaboratif de l'équipe.

Annexe 3 : Base de Données Vectorielle (PostgreSQL / pgvector)

The screenshot shows the pgAdmin 4 interface. The left pane displays the 'Object Explorer' with a tree view showing the database structure. The main pane shows the 'Query Editor' with a SQL query that creates a table named 'embeddings' with a primary key 'id' and a 'vector' column. Below the query, the 'Data Output' pane shows the results of the query, displaying a list of text passages and their corresponding vector representations.

id	text	vector
1	Sacerdos' Relived Ordeal	[-0.017467184,-0.083847545,-0.038951848,-0.017354019,0.019696614,-0.014950157,0.02144781,0.07875104,-0.06650161,-0.03455417,-0.002489321,0.026873624,-0.020563014,-0.01023893,-
2	Scholar Lost in Enrapture	[-0.03751328,-0.0248327,-0.008211421,-0.052721474,0.007026503,-0.051346444,0.056662366,0.1425158,-0.05306731,0.010218035,0.028326867,0.014693761,0.026810719,0.035006355,-0.01
3	The Wind-Scouring Valorous	[-0.14618777,0.029497745,-0.06137857,0.0037354338,0.06595803,-0.0324221,0.051367838,0.0975657,0.015637638,0.013001312,0.046619598,0.02610296,0.027995462,-0.030226327,-0.04466
4	Iron Cavalry Against the Scourge	[-0.09552664,-0.030078655,0.08243766,-0.02410156,0.09591409,-0.024043659,0.036317077,0.10353636,-0.057169467,0.06881431,0.022799687,-0.0308417,-0.0069715837,-0.0024722351,0.0
5	Watchmaker, Master of Dream Machinations	[-0.10061123,-0.037127215,0.040142946,-0.008991735,0.046497703,0.018913228,-0.0661103,0.087174416,-0.045449685,0.050120123,0.050638538,-0.016638438,-0.03288588,0.028422114,
6	Pioneer Diver of Dead Waters	[-0.1061423,-0.0074606105,0.050084997,-0.034070194,0.0506594,-0.080397926,0.040266573,0.085344076,-0.039914894,0.016953537,0.018199502,-0.026913607,0.037480105,-0.017759377,-
7	Prisoner in Deep Confinement	[-0.11287476,-0.01104738,0.0041587194,0.01367851,0.06772128,-0.066797,0.10727208,0.07411573,-0.032124825,0.021844178,0.03612571,-0.0006331399,0.039435122,-0.047839615,-0.0211
8	The Ashblazing Grand Duke	[-0.14775746,-0.0100579085,0.017027002,0.05624755,-0.004378877,-0.043937914,0.03498508,0.049996087,0.048856802,-0.008534413,0.06808504,0.070647255,0.014244043,-0.043236744,
9	Messenger Traversing Hackerspace	[-0.10143094,-0.061148398,-0.032638527,0.010728497,0.012351326,0.01849827,-0.023585888,0.05072568,-0.03725495,0.005308512,0.04303343,0.06045402,-0.00987718,-0.0037603062,0.03
10	Longevity Disciple	[-0.10388135,-0.02784939,-0.01882234,-0.015205044,-0.009271648,-0.014008686,-0.026853312,0.07871245,-0.050344445,-0.041420672,0.070161626,-0.004901482,0.012105643,-0.00823495,
11	Passerby of Wandering Cloud	[-0.08183868,-0.013744844,0.0027253798,0.04049397,-0.025261955,0.022728523,0.08731592,0.021403212,-0.024797956,-0.06340836,0.0455343,0.04311147,-0.004512614,-0.004125684,0.02
12	Muskeeter of Wild Wheat	[-0.11097237,-0.028922716,-0.076811306,-0.014875353,0.04830012,-0.04164871,0.02174451,0.0949544,-0.04705477,0.026707355,0.008810818,0.046525415,-0.025153479,-0.033498064,0.02
13	Hunter of Glacial Forest	[-0.14048293,-0.029854119,0.005816729,-0.0036455453,0.026699634,-0.056465037,0.06680103,0.1529622,-0.03877095,0.017426254,0.0068212696,-0.016201431,0.029385157,-0.007929569,0.0
14	Knight of Purity Palace	[-0.10615234,1.32103939,-0.037406508,0.007782971,0.048776437,-0.06641035,0.06553043,0.09313388,-0.04693607,0.0178203,0.08871045,-0.012419403,0.01635685,0.07562154,0.0201
15	Bland of Sizzling Thunder	[-0.13421956,0.023476418,0.003637288,0.024672829,-0.0085541345,-0.021089092,0.108701244,0.09131911,-0.010128845,-0.011080186,0.049037393,0.023703536,-0.005040087,-0.00983
16	Champion of Streetwise Boxing	[-0.14220591,0.01770257,0.004611471,0.04194723,0.037654847,-0.025145626,0.051854707,0.10356605,0.00012724719,0.016874467,0.04292906,0.0013371826,0.014326731,-0.0247719395,-
17	Thief of Shooting Meteor	[-0.095095364,0.026458556,0.034110318,0.034238145,0.087823175,0.01174434,0.011667693,0.047030494,-0.02640518,0.03073375,0.0073340535,-0.034831267,-0.03488889,0.0062398,0.0
18	Guard of Wuthering Snow	[-0.1639712,0.07998245,0.051664617,0.06215817,-0.004962286,-0.035275158,0.029948022,0.0031025393,-0.06379439,-0.020713264,0.025142917,0.06032961,-0.02729799,-0.009048634,0.0

FIGURE 3 – Visualisation (via pgAdmin) de la table stockant les passages de texte et leurs vecteurs (embeddings) associés dans PostgreSQL avec pgvector, confirmant le bon fonctionnement de l'indexation pour la recherche sémantique (Test FP-04).

Annexe 4 : Client de Test API (Postman)

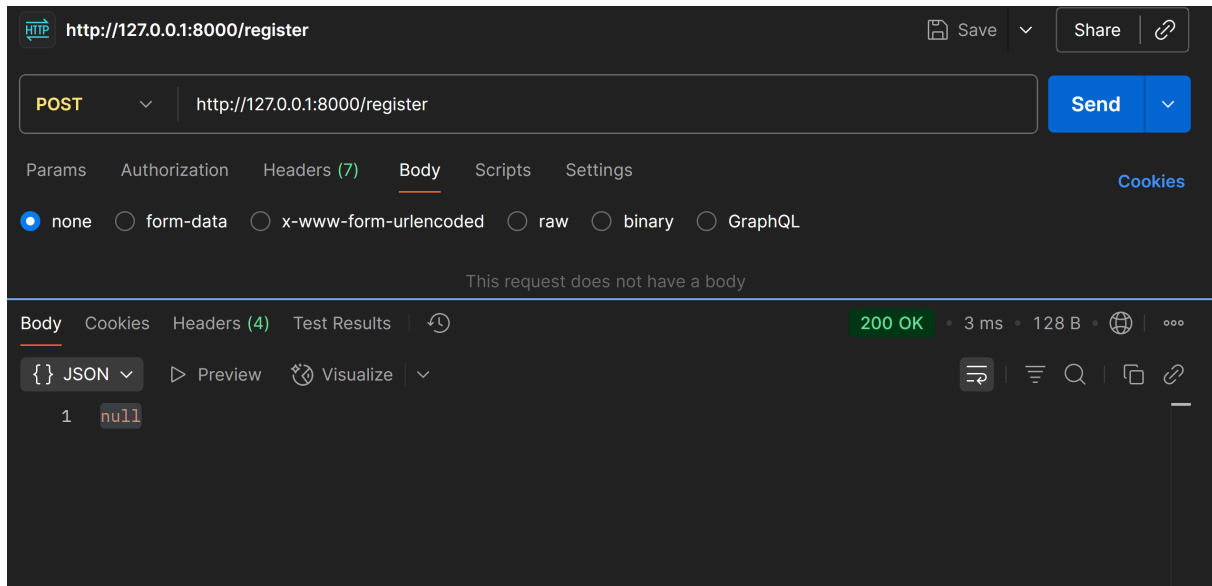


FIGURE 4 – Interface Postman validant le bon fonctionnement des endpoints locaux (ici /register retournant un statut 200 OK), un outil indispensable pour tester les requêtes API de manière isolée avant l’intégration du frontend.

10 Glossaire

- **API (Application Programming Interface)** : Interface de programmation permettant à différentes applications informatiques de communiquer et d’échanger des données entre elles.
- **Backend** : Partie invisible d’une application côté serveur, chargée de la logique métier, du traitement des données et de la communication avec la base de données.
- **Cas limite (ou Edge case)** : Scénario de test impliquant des conditions extrêmes ou inattendues pour évaluer la robustesse du système.
- **Cas nominal** : Scénario de test où l’utilisateur se comporte exactement comme prévu, sans générer d’erreur.
- **Chatbot** : Programme informatique conçu pour simuler une conversation avec des utilisateurs humains via une interface textuelle.
- **Framework** : Ensemble d’outils et de composants logiciels structurés servant de base pour faciliter et accélérer le développement d’une application.
- **Inférence** : Phase durant laquelle un modèle d’intelligence artificielle préalablement entraîné est mis en production pour analyser de nouvelles requêtes et générer des réponses.
- **Livrable** : Document, application ou composant technique qui doit être fourni contractuellement à une date précise lors du déroulement d’un projet.
- **Maîtrise d’œuvre** : Équipe technique chargée de la conception, du développement et de la réalisation globale du projet.

- **Maîtrise d'ouvrage** : Personne physique ou morale pour le compte de laquelle le projet est réalisé et qui en définit les besoins et les objectifs.
- **ORM (Object-Relational Mapping)** : Technique de programmation permettant de manipuler les données d'une base de données classique en utilisant directement les concepts du langage de programmation, sans avoir à écrire de requêtes SQL manuellement.
- **Prompt système** : Instruction initiale invisible pour l'utilisateur final, fournie à une intelligence artificielle pour paramétrer son rôle, son ton et ses restrictions de comportement.
- **SVN (Subversion)** : Système de gestion de versions permettant de sauvegarder l'historique du code source et de faciliter le travail simultané de plusieurs développeurs.
- **UML (Unified Modeling Language)** : Langage de modélisation graphique standardisé, utilisé pour concevoir et schématiser l'architecture d'un système logiciel avant son développement.

Références

- [1] *pytest*. Documentation officielle du framework de tests en Python.
Disponible sur : <https://docs.pytest.org/>
- [2] *Postman*. Documentation officielle de la plateforme de test d'API.
Disponible sur : <https://learning.postman.com/docs/>
- [3] *Site web de Ollama*.
Disponible sur : <https://ollama.com/>
- [4] *Pléiade* (Serveurs de l'université).
Disponible sur : <https://pleiade.mi.parisdescartes.fr/>
- [5] *La Forge* (Projets).
Disponible sur : <https://forge.ens.math-info.univ-paris5.fr/projects/>
- [6] *Site web de Chainlit*.
Disponible sur : <https://chainlit.io/>
- [7] *Google Gemini*. Pour la recherche générale et la rédaction sur Overleaf
Disponible sur : <https://gemini.google.com/>

Index

- API, 2, 6, 12, 13
- Assistant Pédagogique, 2, 4
- Base de données vectorielle, 2, 6, 8
- Cahier de recettes, 2, 3, 5, 6
- Cahier des charges, 2, 5, 6, 10
- Chainlit, 6, 8, 10, 13
- curl, 6
- Dataset de requêtes, 4, 6
- Embeddings, 4, 7–9, 11
- Filtrage, 2, 6, 8, 10
- Forge, 4, 6, 10, 11, 13
- Hallucination, 2, 7
- Inférence, 6, 12
- Interface web, 3, 4, 6, 8–10
- Livable, 4, 9, 12
- LLM/SLM, 3, 7–9
- Maîtrise d'œuvre, 3, 12
- Maîtrise d'ouvrage, 2, 3, 13
- Ollama, 6, 8, 13
- ORM, 2, 13
- pgvector, 6, 8, 11
- Pléiade, 2, 6, 8, 13
- PostgreSQL, 6, 11
- Postman, 6, 12, 13
- Prompt système, 4, 9, 13
- pytest, 6, 13
- RAG (Retrieval-Augmented Generation), 3, 6, 10
- SVN, 13
- SVN (Subversion), 4, 6, 10, 11
- Tests d'intégration, 2, 6
- Tests fonctionnels, 2, 6
- Tests unitaires, 2, 6
- UML, 13
- UML (Unified Modeling Language), 5, 10
- uv (package manager), 6
- Vectorisation, 2, 4, 10